

Министерство науки и высшего образования Российской Федерации

Федеральное государственное автономное образовательное
учреждение высшего образования
Национальный исследовательский Нижегородский государственный
университет им. Н.И. Лобачевского

Институт информационных технологий, математики и механики

Отчет по лабораторной работе №3
**«Решение СЛАУ методом Гаусса с выбором ведущего
элемента»**

Выполнил:

студент группы 382561-ПМии1
Сазонов Павел Игоревич

Проверила:

Бусько П.В.

Содержание

Введение	3
Постановка задачи.....	4
Руководство пользователя	5
Описание программной реализации	6
Результаты экспериментов	14
Заключение.....	18
Литература	19
Приложение	20

Введение

Метод Гаусса является одним из основных прямых методов решения систем линейных алгебраических уравнений (СЛАУ). Он широко применяется в вычислительной математике, физике, инженерии и экономике благодаря своей универсальности и относительной простоте реализации.

Однако классический метод Гаусса без выбора ведущего элемента может оказаться неработоспособным при наличии нулевых элементов на главной диагонали. Для решения этой проблемы применяется модификация метода — метод Гаусса с выбором ведущего элемента (partial pivoting), который обеспечивает численную устойчивость алгоритма.

В данной работе реализован метод Гаусса с выбором ведущего элемента для решения СЛАУ с действительной квадратной матрицей. Реализация выполнена на языке C++ с использованием шаблонных классов.

Постановка задачи

Цель работы: реализовать метод Гаусса для решения системы линейных алгебраических уравнений с действительной квадратной матрицей, использующий выбор ведущего элемента.

Требования к реализации:

1. Реализовать шаблонный класс Vector для работы с одномерными динамическими массивами.
2. Реализовать класс квадратной матрицы Matrix, который является наследником шаблона Vector<Vector<T>>.
3. Реализовать класс SLAE (система линейных алгебраических уравнений), наследующий Matrix, содержащий метод gaussMethod, принимающий правую часть в качестве аргумента и возвращающий решение в виде вектора.
4. Реализовать выбор ведущего элемента (partial pivoting) для обеспечения работы с матрицами, содержащими нулевые диагональные элементы.
5. Реализовать функцию swar для перестановки строк матрицы.

Исходные данные:

- Квадратная матрица коэффициентов A размера $n \times n$
- Вектор правой части b размера n

Результат работы:

- Вектор решения x размера n, удовлетворяющий уравнению $A \cdot x = b$

Руководство пользователя

Программа реализована в консольном режиме и взаимодействует с пользователем через стандартный поток ввода/вывода.

Запуск программы

После компиляции и запуска программа последовательно запрашивает:

1. **Размер системы** — целое положительное число n
2. **Элементы матрицы A** — n строк по n действительных чисел
3. **Элементы вектора b** — n действительных чисел

Пример работы программы (рис. 1)

```
Enter matrix size:
3
Enter matrix:
0 1 1
1 0 1
1 1 0
Enter right path:
2 5 3
Solution:
[ 3 0 2 ]
```

Рис. 1 (Пример работы программы)

Сообщения об ошибках

Сообщение	Причина
Error: Right part size doesn't match matrix size!	Размер правой части не совпадает с размером матрицы
Error: Matrix is singular!	Матрица вырождена, решение не существует или не единственно
Out of range!	Попытка обращения к элементу за пределами массива

Описание программной реализации

Проект реализован в виде одного файла `main.cpp`, содержащего все необходимые классы. Выделение в отдельные файлы (`.h` и `.cpp`) для шаблонных классов нецелесообразно, так как шаблоны нельзя скомпилировать отдельно — они собираются только в момент использования.

Иерархия классов

Класс `Matrix` наследует `Vector<Vector<T>>`, класс `SLAE` наследует `Matrix<T>`

Класс `Vector`

Шаблонный класс для работы с одномерным динамическим массивом.

Поля класса

Таблица 1 – Поля класса `Vector`

Поле	Тип	Описание
<code>arr</code>	<code>T*</code>	Указатель на динамический массив элементов типа <code>T</code>
<code>size</code>	<code>size_t</code>	Размер вектора (количество элементов)

Методы класса

Таблица 2 - Методы класса `Vector`

Метод	Параметры	Описание
<code>Vector(size_t size_ = 1, T value = 1)</code>	<code>size_</code> - размер вектора, <code>value</code> - значение для заполнения	Конструктор по умолчанию. Создает вектор размера <code>size_</code> и заполняет его значением <code>value</code>
<code>Vector(const Vector& other)</code>	<code>other</code> - ссылка на другой вектор	Конструктор копирования. Выполняет копирование данных
<code>~Vector()</code>	нет	Деструктор. Освобождает динамически выделенную память

Операторы

Таблица 3 – Перегрузка операторов класса Vector

Оператор	Параметры	Возвращаемое значение	Описание
<code>operator=</code>	<code>const Vector& other</code>	<code>Vector&</code>	Оператор присваивания. Реализует копирование с защитой от самоприсваивания
<code>operator[]</code>	<code>size_t i</code>	<code>T&</code>	Доступ к элементу по индексу (неконстантная версия). При выходе за границы выводит сообщение об ошибке
<code>operator[]</code>	<code>size_t i</code>	<code>const T&</code>	Доступ к элементу по индексу (константная версия). Используется для const-объектов
<code>operator<< (друг)</code>	<code>ostream& os, const Vector& V</code>	<code>ostream&</code>	Вывод вектора в формате [a1 a2 a3 ...]

Оператор	Параметры	Возвращаемое значение	Описание
<code>operator>></code> (друг)	<code>istream& is,</code> <code>Vector& V</code>	<code>istream&</code>	Ввод элементов вектора из потока

Дополнительные методы

Таблица 4 – Дополнительные методы класса `Vector`

Метод	Возвращаемое значение	Описание
<code>getS()</code> <code>const</code>	<code>size_t</code>	Возвращает размер вектора. Метод помечен <code>const</code> , так как не изменяет объект

Класс `Matrix`

Класс квадратной матрицы, реализованный как вектор строк, где каждая строка является вектором. Наследует `Vector<Vector<T>>`, что позволяет использовать все методы базового класса.

Наследование

```
class Matrix : public Vector<Vector<T>>
```

Таким образом, матрица представляет собой одномерный массив, каждый элемент которого — это вектор (строка матрицы).

Конструктор

Таблица 5 – Конструктор класса `Matrix`

Метод	Параметры	Описание
<code>Matrix(size_t size_ = 1, T value = 1)</code>	<code>size_</code> - размер матрицы (<code>size_ × size_</code>), <code>value</code> - значение для заполнения	Создает квадратную матрицу размера <code>size_ × size_</code> . Все элементы инициализируются значением <code>value</code>

Особенность: конструктор вызывает конструктор родительского класса с параметрами (size_, Vector<T>(size_, value)), то есть создает вектор из size_ строк, каждая из которых является вектором размера size_, заполненным значением value.

Методы класса

Таблица 6 – Метод swap в классе Matrix

Метод	Параметры	Описание
swap(size_t row1, size_t row2)	row1, row2 - индексы строк	Меняет местами две строки матрицы. Используется при выборе ведущего элемента в методе Гаусса

Операторы ввода/вывода

Таблица 7 – Операторы ввода и вывода

Оператор	Описание
operator<< (друг)	Вывод матрицы построчно. Каждая строка выводится с помощью operator<< класса Vector
operator>> (друг)	Ввод матрицы. Последовательно считывает size×size чисел

Класс SLAE

Назначение

Класс системы линейных алгебраических уравнений. Наследует Matrix<T> и добавляет метод решения СЛАУ методом Гаусса с выбором ведущего элемента.

Наследование

```
class SLAE : public Matrix<T>
```

Наследует все поля и методы матрицы, что позволяет обращаться к элементам системы как к элементам матрицы.

Конструктор

Таблица 8 – Конструктор класса SLAE

Метод	Параметры	Описание
SLAE(size_t size_ = 1, T value_ = 0)	size_ - размер системы, value_ - значение для заполнения	Создает систему с матрицей размера size_ $\times$ size_, заполненной значением value_

Основной метод: gaussMethod

Прототип функции

Vector<T> gaussMethod(const Vector<T>& rightPart)

Параметры

Принимает константную ссылку на вектор значений правой части системы линейных уравнений.

Возвращаемое значение

Vector<T> — вектор решения системы.

Алгоритм работы

Шаг 1. Подготовка

```
Matrix<T> A = *this; // копия матрицы системы
```

```
Vector<T> b = rightPart; // копия правой части
```

```
size_t n = this->size; // размер системы
```

Шаг 2. Проверка размеров

```
if (b.getS() != n) {  
    cerr << "Error: Right part size doesn't match matrix size!\n";  
    return Vector<T>(n, 0);  
}
```

Шаг 3. Основной цикл (по столбцам)

```
for (int k = 0; k < n; k++) {  
    // обработка k-го столбца
```

```
}
```

Шаг 3.1. Выбор ведущего элемента

Назначение: найти строку с максимальным по модулю элементом в k-м столбце (начиная с k-й строки). Это обеспечивает численную устойчивость и позволяет работать с нулевыми диагональными элементами.

```
int maxRow = k;

T maxAbs = std::abs(A[k][k]);

for (int i = k + 1; i < n; i++) {

    if (std::abs(A[i][k]) > maxAbs) {

        maxAbs = std::abs(A[i][k]);

        maxRow = i;

    }

}
```

Шаг 3.2. Проверка на вырожденность

```
if (maxAbs < 1e-10) {

    cerr << "Error: Matrix is singular!\n";

    return Vector<T>(n, 0);

}
```

Шаг 3.3. Перестановка строк (при необходимости)

```
if (maxRow != k) {

    A.swap(k, maxRow); // меняем строки в матрице

    swap(b[k], b[maxRow]); // меняем соответствующие элементы в правой части

}
```

Шаг 3.4. Нормировка текущей строки

Делим k-ю строку матрицы и k-й элемент правой части на ведущий элемент $\text{pivot} = A[k][k]$:

```
T pivot = A[k][k];

for (int j = k; j < n; j++) {
```

```

    A[k][j] /= pivot;
}

```

```

b[k] /= pivot;

```

Шаг 3.5. Исключение остальных строк

Для всех строк $i \neq k$ вычитаем из i -й строки k -ю строку, умноженную на $\text{factor} = A[i][k]$:

```

for (int i = 0; i < n; i++) {
    if (i != k) {
        T factor = A[i][k];
        if (std::abs(factor) > 1e-12) {
            for (int j = k; j < n; j++) {
                A[i][j] -= factor * A[k][j];
            }
            b[i] -= factor * b[k];
        }
    }
}

```

Шаг 4. Возврат решения

После завершения цикла матрица A становится единичной, а вектор b содержит решение системы. Возвращаем b .

Основная функция main

```

int main() {
    int n;

    cout << "Enter matrix size: ";

    cin >> n;

    SLAE<double> a(n, 0);

    Vector<double> b(n, 0);

    cout << "Enter matrix A (" << n << "x" << n << "):\n";

    cin >> a;
}

```

```
cout << "Enter vector b (" << n << " elements): ";  
  
cin >> b;  
  
Vector<double> ans = a.gaussMethod(b);  
  
cout << "\nSolution:\n" << ans << endl;  
  
return 0;  
  
}
```

Последовательность работы:

1. Запрос размера системы у пользователя
2. Создание объекта SLAE с заданным размером
3. Ввод матрицы коэффициентов
4. Ввод вектора правой части
5. Решение системы методом Гаусса
6. Вывод результата

Результаты экспериментов

Эксперимент 1. Матрица с нулевым диагональным элементом (проверка выбора ведущего элемента)

Цель эксперимента: проверить работу выбора ведущего элемента на матрице, где первый диагональный элемент равен нулю.

Условия:

$$A = \begin{bmatrix} 0 & 1 & 1 \\ 1 & 0 & 1 \\ 1 & 1 & 0 \end{bmatrix} \quad b = \begin{bmatrix} 2 \\ 3 \\ 4 \end{bmatrix}$$

$$\begin{bmatrix} 1 & 0 & 1 \end{bmatrix} \quad \begin{bmatrix} 3 \end{bmatrix}$$

$$\begin{bmatrix} 1 & 1 & 0 \end{bmatrix} \quad \begin{bmatrix} 4 \end{bmatrix}$$

Данная матрица является невырожденной (определитель = 2), однако классический метод Гаусса без выбора ведущего элемента на первом же шаге сталкивается с нулевым элементом на диагонали, что делает его неприменимым.

Результат работы программы:

```
Enter matrix size:
3
Enter matrix:
0 1 1
1 0 1
1 1 0
Enter right path:
2 3 4
Solution:
[ 2.5 1.5 0.5 ]
```

Рис. 2 (Результаты первого эксперимента)

Анализ результата:

- Программа обнаружила нулевой ведущий элемент в позиции (0,0)
- Выполнена перестановка строк 0 и 1

Вывод: выбор ведущего элемента успешно справляется с матрицами, имеющими нулевые диагональные элементы.

Эксперимент 2. Решение системы с известным ответом

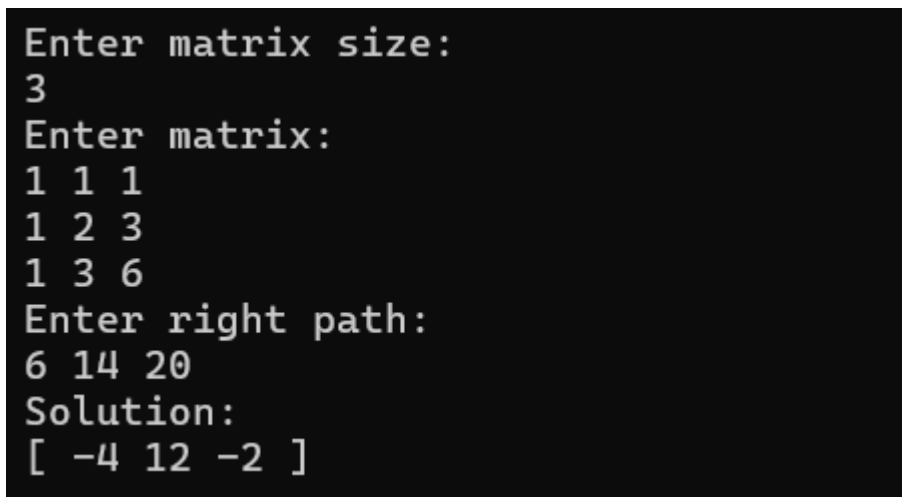
Цель эксперимента: проверить корректность работы метода на системе с целочисленным решением.

Условия:

$$A = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 2 & 3 \\ 1 & 3 & 6 \end{bmatrix} \quad b = \begin{bmatrix} 6 \\ 14 \\ 20 \end{bmatrix}$$

Ожидаемое решение: $x = [-4, 12, -2]$

Результат работы программы:



```
Enter matrix size:
3
Enter matrix:
1 1 1
1 2 3
1 3 6
Enter right path:
6 14 20
Solution:
[ -4 12 -2 ]
```

Рис. 3 (Результаты второго эксперимента)

Анализ результата: Погрешность отсутствует, так как решение является целочисленным и точно представимо в типе double.

Эксперимент 3. Решение системы с дробным решением

Цель эксперимента: проверить работу метода с типом double на системе, дающей дробное решение.

Условия:

$$A = \begin{bmatrix} 2 & 1 \\ 1 & 3 \end{bmatrix} \quad b = \begin{bmatrix} 5 \\ 5 \end{bmatrix}$$

Ожидаемое решение: $x = [2, 1]$

Результат работы программы:

```
Enter matrix size:
2
Enter matrix:
2 1
1 3
Enter right path:
5 5
Solution:
[ 2 1 ]
```

Рис. 4 (Результаты третьего эксперимента)

Эксперимент 4. Проверка обработки вырожденной матрицы

Цель эксперимента: проверить корректность обработки случая, когда матрица системы является вырожденной или несовместной.

Условия:

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 2 & 4 & 6 \\ 3 & 6 & 9 \end{bmatrix} \quad b = \begin{bmatrix} 1 \\ 3 \\ 3 \end{bmatrix}$$

Вторая строка является удвоенной первой, третья — утроенной. При этом правые части не согласованы (1, 3, 3), что делает систему несовместной.

Результат работы программы:

```
Enter matrix size:
3
Enter matrix:
1 2 3
2 4 6
3 6 9
Enter right path:
1 3 3
Error: Matrix is singular!
Solution:
[ 0 0 0 ]
```

Рис. 5 (Результаты четвертого эксперимента)

Анализ результата: программа корректно распознала вырожденную матрицу, выдала сообщение об ошибке и вернула нулевой вектор, не пытаясь продолжить вычисления.

Заключение по результатам

Проведенные эксперименты подтверждают:

1. Алгоритм корректно решает системы с невырожденными матрицами
2. Выбор ведущего элемента позволяет обрабатывать матрицы с нулевыми диагональными элементами
3. Алгоритм корректно определяет вырожденные матрицы и не пытается их решать
4. Использование шаблонного класса позволяет работать с различными типами данных (int, double)

Заключение

В ходе выполнения лабораторной работы были решены следующие задачи:

1. Реализован шаблонный класс `Vector<T>` для работы с динамическими массивами.
2. Реализован класс квадратной матрицы `Matrix` как наследник `Vector<Vector<T>>`.
3. Реализован класс `SLAE` с методом Гаусса, включающим выбор ведущего элемента.
4. Проведено тестирование на различных матрицах.

Выводы:

- Метод Гаусса с выбором ведущего элемента позволяет решать СЛАУ с матрицами, имеющими нулевые диагональные элементы.
- Код является шаблонным, что позволяет использовать его с различными числовыми типами (`int`, `double`, `float`).

Литература

1. Керниган Б., Ритчи Д., Фьюэр А. Язык программирования СИ //М.: Финансы и статистика. – 1992.
2. Кнут Д. Э. Искусство программирования: Сортировка и поиск. – Издательский дом Вильямс, 2000. – Т. 3.

Приложение

```
Vector<T> gaussMethod(const Vector<T>& rightPart) {
    Matrix<T> A = *this;
    Vector<T> b = rightPart;
    size_t n = this->size;

    if (b.getS() != n) {
        cerr << "Error: Right part size doesn't match matrix
size!\n";
        return Vector<T>(n, 0);
    }

    for (int k = 0; k < n; k++) {
        // Поиск максимального элемента в столбце
        int maxRow = k;
        T maxAbs = std::abs(A[k][k]);
        for (int i = k + 1; i < n; i++) {
            if (std::abs(A[i][k]) > maxAbs) {
                maxAbs = std::abs(A[i][k]);
                maxRow = i;
            }
        }

        if (maxAbs < 1e-10) {
            cerr << "Error: Matrix is singular!\n";
            return Vector<T>(n, 0);
        }

        if (maxRow != k) {
            A.swap(k, maxRow);
            swap(b[k], b[maxRow]);
        }

        T pivot = A[k][k];
        for (int j = k; j < n; j++) A[k][j] /= pivot;
        b[k] /= pivot;
    }
}
```

```

        for (int i = 0; i < n; i++) {
            if (i != k && std::abs(A[i][k]) > 1e-12) {
                T factor = A[i][k];
                for (int j = k; j < n; j++) A[i][j] -= factor *
A[k][j];
                b[i] -= factor * b[k];
            }
        }
    }
    return b;
}

```